ELSEWHERE / PROJECT GUIDE

Travel Planner

A project overview, architecture map, and publishing guide

Updated 15 October 2026

What the project does

Elsewhere is a React and Vite travel-planning website connected to a FastAPI API. The API invokes the existing LangGraph workflow to gather flight-feed records, accommodation web references, and an AI-generated itinerary. PostgreSQL stores LangGraph checkpoints.

At a glance

FRONTEND	API	ORCHESTRATION
React 19, Vite 7, lucide-react, react-markdown	FastAPI, Pydantic request validation, CORS	LangGraph agents, sequential graph, PostgreSQL checkpointer

The user experience

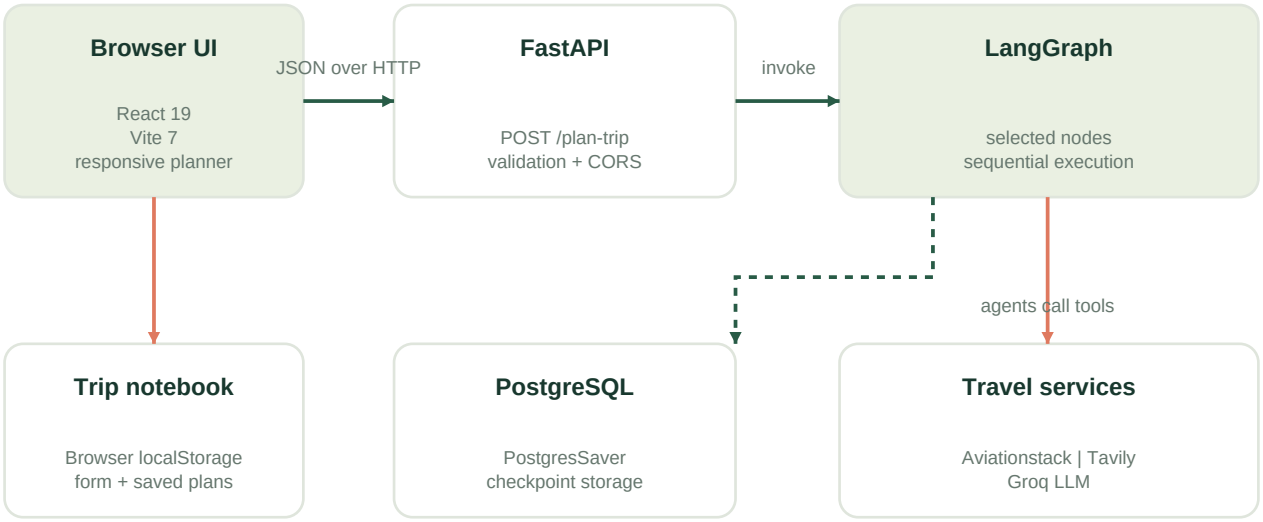
- Explore destination photography and select one of 11 featured destinations or search for a place.
- Edit departure, destination, dates, traveller count, USD budget, and travel preferences.
- Request Flights, Hotels, Itinerary, or a Complete Trip Plan. Focused choices invoke only selected agent nodes.
- Review readable Markdown itineraries, source disclosures, loading/error states, and the budget target.
- Trip form data and completed plans are saved in this browser using localStorage.

Important distinction

The complete plan is sequential, not parallel. Focused options run only their selected node. Flight records are not route/date filtered and have no fare feed; hotel data is web research, not live availability. AI itinerary expenses are estimates, not quotes.

1. System architecture

The browser calls one FastAPI route. The API chooses an existing full or focused LangGraph path.



One checkpointer is opened at API startup and reused by planner graph runs.

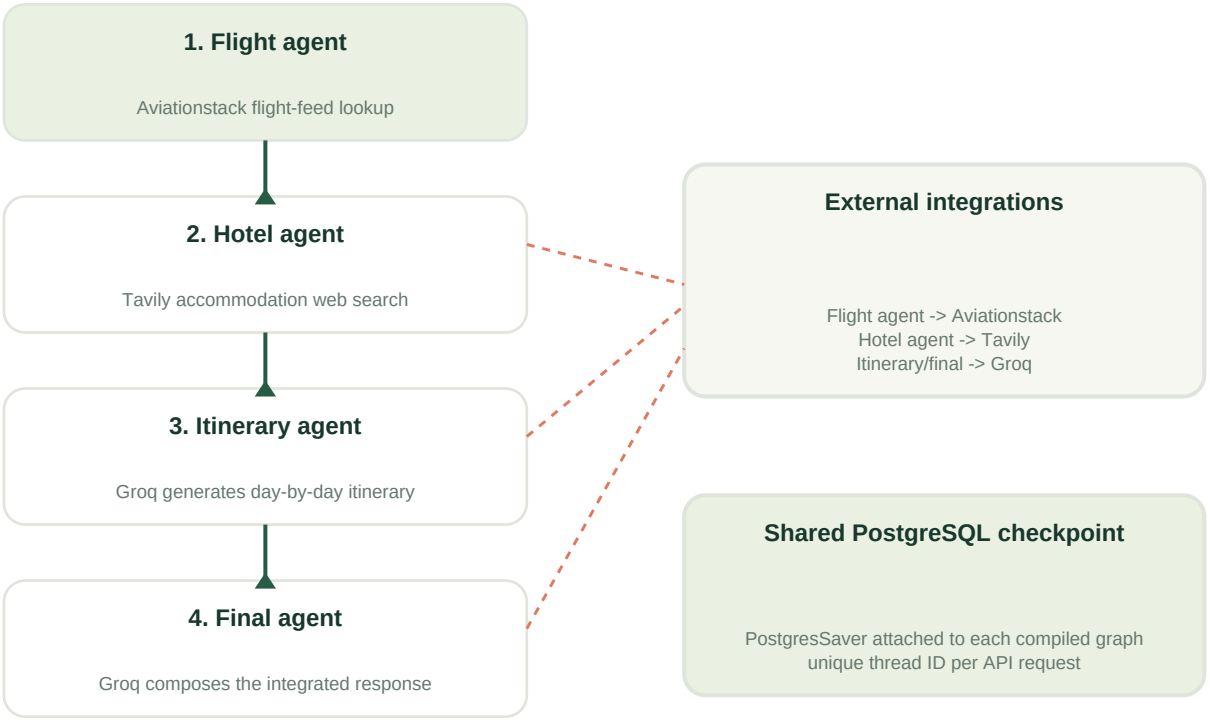
Trust boundaries and responsibilities

Component	Responsibility
React / Vite	Destination exploration, trip form, planner selection, result display, browser-local saved trips.
FastAPI	POST /plan-trip validation, allowed-origin CORS, graph invocation, response shaping, explicit 422/502 errors.
LangGraph	Runs selected travel agents in a defined sequence; complete mode also invokes the final response agent.
PostgreSQL	Stores graph checkpoints via PostgresSaver. The API opens the connection during lifespan startup.
External services	Aviationstack feed, Tavily web search, and Groq chat model called by existing tools/agents.

The backend listens locally on port 8000 by default; Vite serves the frontend on port 5173. VITE_API_URL is a frontend build-time setting. API credentials stay in the root backend .env.

2. LangGraph workflow

The existing TravelState carries the user query, messages, flight results, hotel results, itinerary, and LLM call count.



Full plan: all four nodes, in this fixed order.

Focused plans compile only their requested agent node(s); none run in parallel.

Execution modes

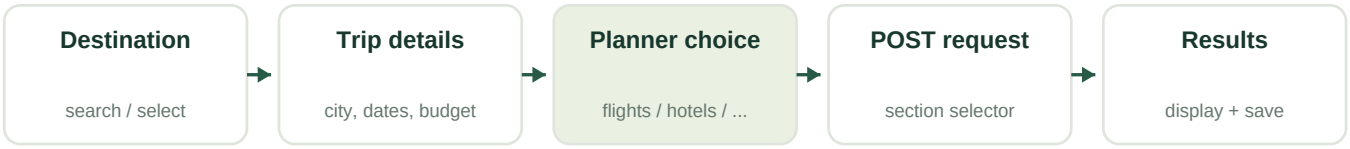
User selection	Graph path executed
Flights	flight_agent
Hotels	hotel_agent
Itinerary	itinerary_agent
Complete trip plan (or legacy request)	flight_agent -> hotel_agent -> itinerary_agent -> final_agent

Checkpointing

connect_database() constructs and sets up PostgresSaver when FastAPI starts. The API reuses that checkpointer for the default graph and for graphs compiled with selected sections. Each request receives a UUID thread_id. PostgreSQL persists LangGraph checkpoints; the UI's My Trips notebook is a separate browser-local feature.

3. Frontend-to-API interaction

The same POST endpoint supports focused planning and the full sequential workflow.



POST /plan-trip request

Field	Requirement / purpose
query	Required string, 2-300 characters; destination context.
duration_days, budget, traveller_count	Required: 1-30 days, positive USD amount, 1-12 travellers.
departure_city, destination_city, destination_country	Optional location details; included by the redesigned UI.
departure_date, return_date	Optional ISO dates; must be supplied together, return cannot precede departure.
preferences	Optional free-text travel preferences (maximum 500 characters).
sections	Optional unique list of flights, hotels, itinerary; defaults to all three.

Response shape

The JSON response returns the normalized trip details, sections, flights[], hotels[], itinerary, and complete_plan. Selective calls leave unrequested sections empty. The complete_plan value is populated for the full selection.

Browser and API behavior

- VITE_API_URL defaults to http://localhost:8000; the root URL can legitimately return 404. Use GET /health to check the API.
- FastAPI CORS defaults to localhost:5173 and 127.0.0.1:5173; configure FRONTEND_ORIGINS in root .env for other origins.
- A planner/tool exception is logged server-side and returned as HTTP 502; schema and date errors are HTTP 422.

4. Code map and integration boundaries

The redesign reuses the existing API and agent implementations rather than creating mock results.

Path	Role
api.py	FastAPI app, CORS, request validation, /health, /plan-trip, output parsing.
main.py	TravelState, flight/hotel/itinerary/final agents, database connection, graph builder.
tools/flight_tool.py	Aviationstack HTTP request and mapping feed records to text.
tools/tavily_tool.py	Tavily accommodation search and source references.
frontend/src/App.jsx	Destination exploration, trip form, planner options, results and My Trips.
frontend/src/styles.css	Responsive layout, colors, cards, feedback states, and animation.
frontend/src/main.jsx	React application entry and stylesheet import.
frontend/package.json	Vite scripts; React, lucide-react and react-markdown dependencies.
requirements.txt	Pinned direct Python packages for the FastAPI/LangGraph backend.
.gitignore	Excludes .env credentials, Python environment, node_modules, and generated frontend build.

What the connected services currently do not supply

• Aviationstack receives a generic flight-feed request (limit 5); code does not filter by route or travel dates.

• Flight fares and reliable durations are unavailable and are labelled as unavailable in the UI.

• Tavily returns web references/snippets, not verified availability, rates, amenities, property photos, or bookings.

• The Groq itinerary is AI-generated; suggested activities and expense figures are estimates, not confirmed prices.

• Live planning needs PostgreSQL and valid Groq, Aviationstack and Tavily credentials. Destination photos load from Unsplash.

Deployment note	This project is configured for local React/Vite + FastAPI use. No Render deployment or Docker configuration is included in the inspected project.

5. Add the project to a portfolio on GitHub

A public GitHub repository makes the source visible. It is also possible for others to download, fork, and clone public repositories; GitHub has no public view-only source setting.

Before uploading

• Install Git for Windows from git-scm.com/download/win, or install GitHub Desktop. Git was not available in the terminal checked for this project.

• Sign in to GitHub and create a new empty public repository named, for example, elsewhere-travel-planner. Do not initialize it with a README, license, or .gitignore; the project already has these files.

• Real backend and frontend environment files are excluded by .gitignore. Do not stage credentials or a database URL containing a password.

First push from PowerShell

```
Set-Location "D:\Sami\Travel Agent Proj"
git init -b main
git status --short
git check-ignore -v .env frontend/.env
git add .
git status --short
git commit -m "Add Elsewhere travel planner"
git remote add origin https://github.com/YOUR-USERNAME/elsewhere-travel-planner.git
git push -u origin main
```

Portfolio presentation ideas

• Add a short project description, feature list, and technology stack to the repository About panel and README.

• Include a homepage screenshot and this project overview PDF in the repository so visitors can understand the UI and architecture quickly.

• Add a live demo link only after you deploy the frontend, API, database, and required integrations.

• Mention honestly that the project uses Groq, Aviationstack, Tavily, and PostgreSQL-backed LangGraph checkpointing.

Credential safety	**Keep .env files private. If any credential is accidentally pushed, rotate/revoke it with its provider; deleting it in a later commit does not erase it from Git history.**